

Fundamentals of Computer Programming

Introduction.

Week 1 | Lecture 1 (1.1)

This Week's Content.

- **Lecture 1.1**
 - Introduction to APS106.
- **Lecture 1.2**
 - Computers, Programs, and The Coding Toolbox.
- **Lecture 1.3**
 - Variables, Expressions, and Operators.

Teaching Team.



Chris
Instructor



Seb
Instructor &
Coordinator



Charlie
TA

Chris.

- Assistant Professor, Materials Science.
- Senior Staff Scientist, University of Toronto's Acceleration Consortium.
- Research focus: Develops and applies computational methods to design, predict, and understand novel/functional materials.



Seb.

- Completed CivMin PhD in 2014 where I studied rock fracture and seismology.
- Senior Director of Product & AI at KORE Geosystems, a mining tech startup.
- Senior Research Scientist at SickKids.
- Co-Founder, 3Lead. 🍃
- Joined UofT CivMin in January 2020.
- Research topics: rock mechanics, ultrasonics, signal processing, computer vision, applied machine learning, and translational engineering.



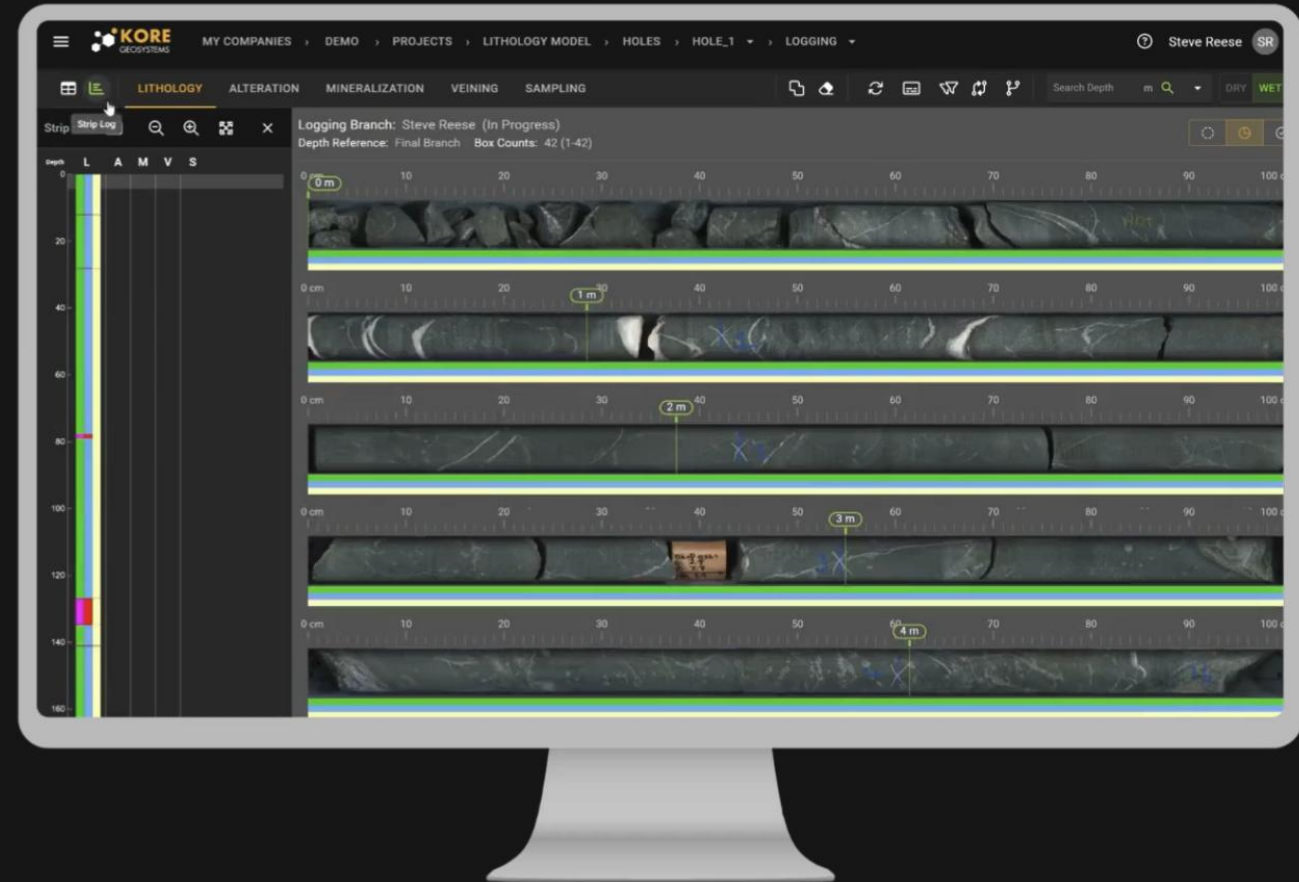
Seb.

- Completed CivMin PhD in 2014 where I studied rock fracture and seismology.
- Senior Director of Product & AI at KORE Geosystems, a mining tech startup.
- Senior Research Scientist at SickKids.
- Co-Founder, 3Lead.
- Joined UofT CivMin in January 2020.
- Research topics: rock mechanics, ultrasonics, signal processing, computer vision, applied machine learning, and translational engineering.



Who is **Kore**?

- **Who we are:** A geologist-first AI company building the next generation of core logging software.
- **What problem we solve:** Core logging teams are overwhelmed by drilling volumes and limited by workforce shortages, leading to massive backlogs, inconsistent data, and costly delays across the mining lifecycle.
- **How we solve it:** We combine high-resolution data, intuitive UX, and geologist-in-the-loop AI to deliver faster, more consistent, and remote-enabled core logging workflows.



WINNER OF

#DISRUPTMINING

PRESENTED BY  GOLDCORP CO-HOSTED BY  KPMG

SUPPORTED BY


NEWMONT
GOLDCORP™

SUPPORTED BY

MAJOR
Drilling

2019 WINNER

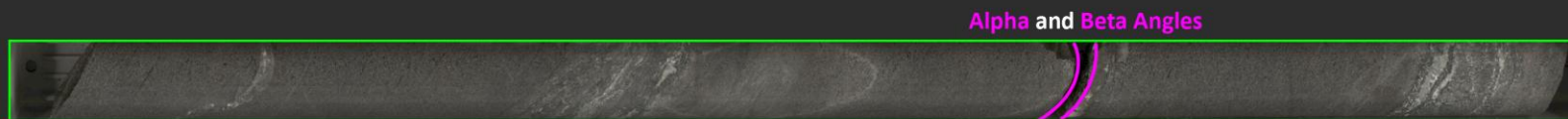
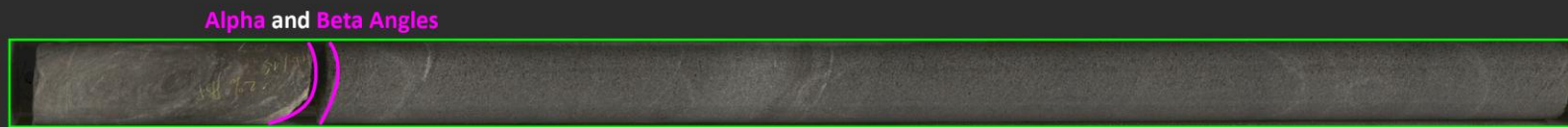
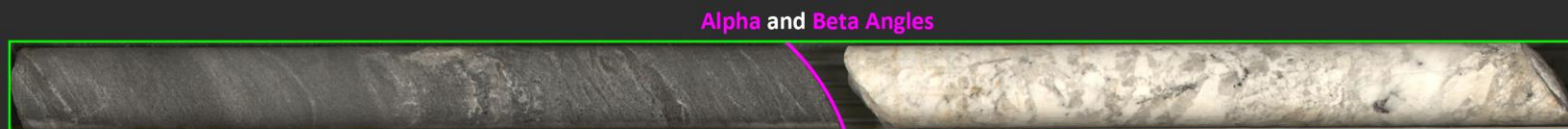
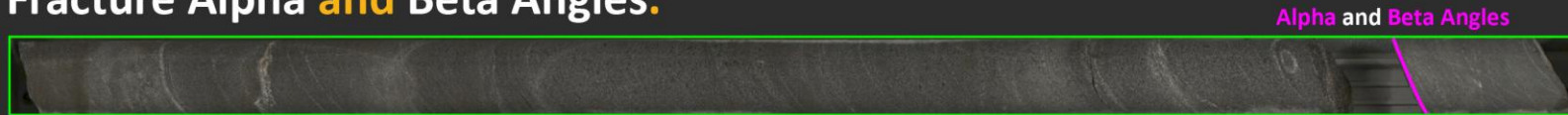
 **GOOD
DESIGN
AWARD**®
WINNER

“NAMES TO KNOW IN MINING”

CIMMAGAZINE

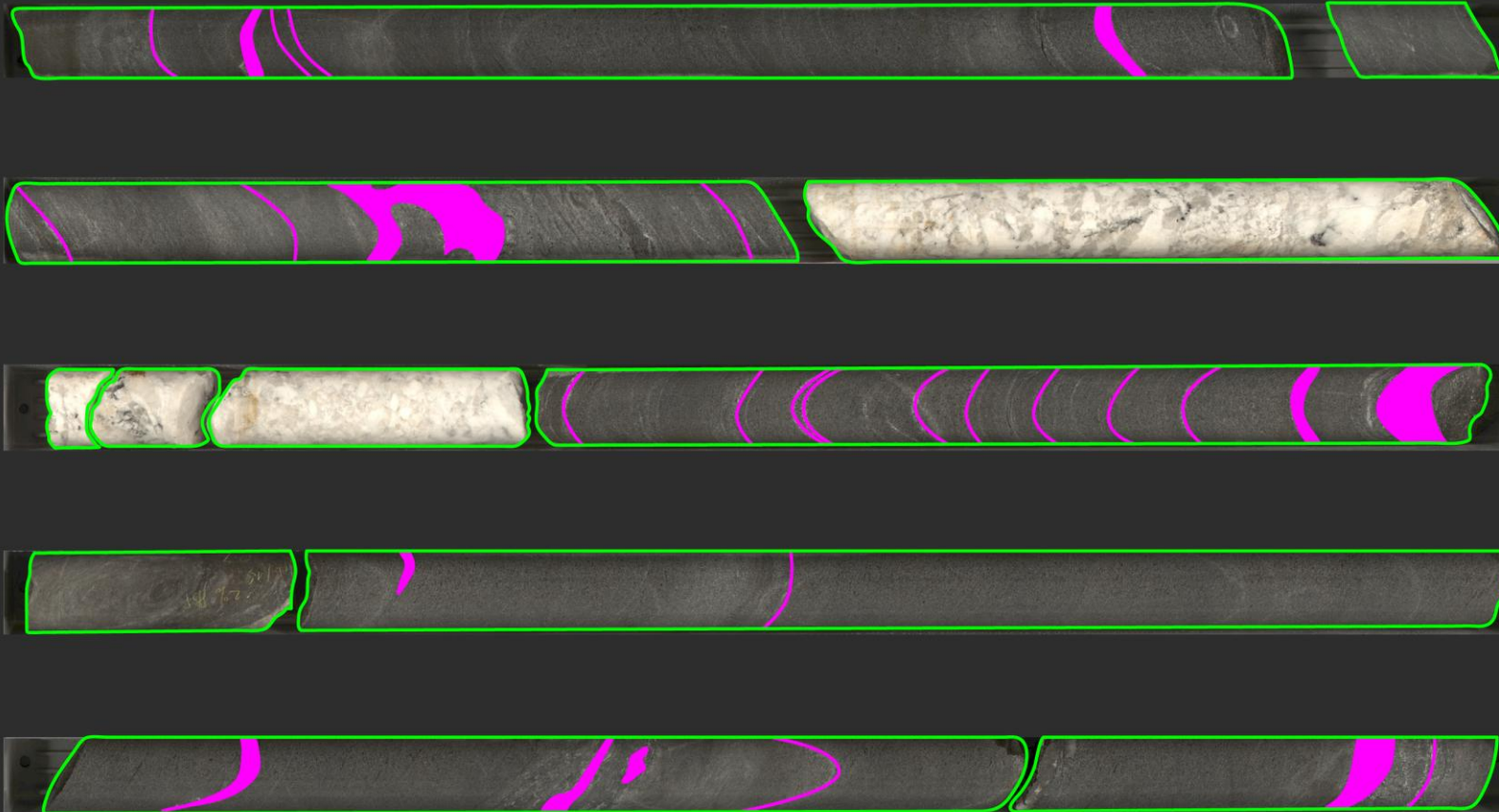
We build AI to... find fractures.

Fracture Alpha and Beta Angles.



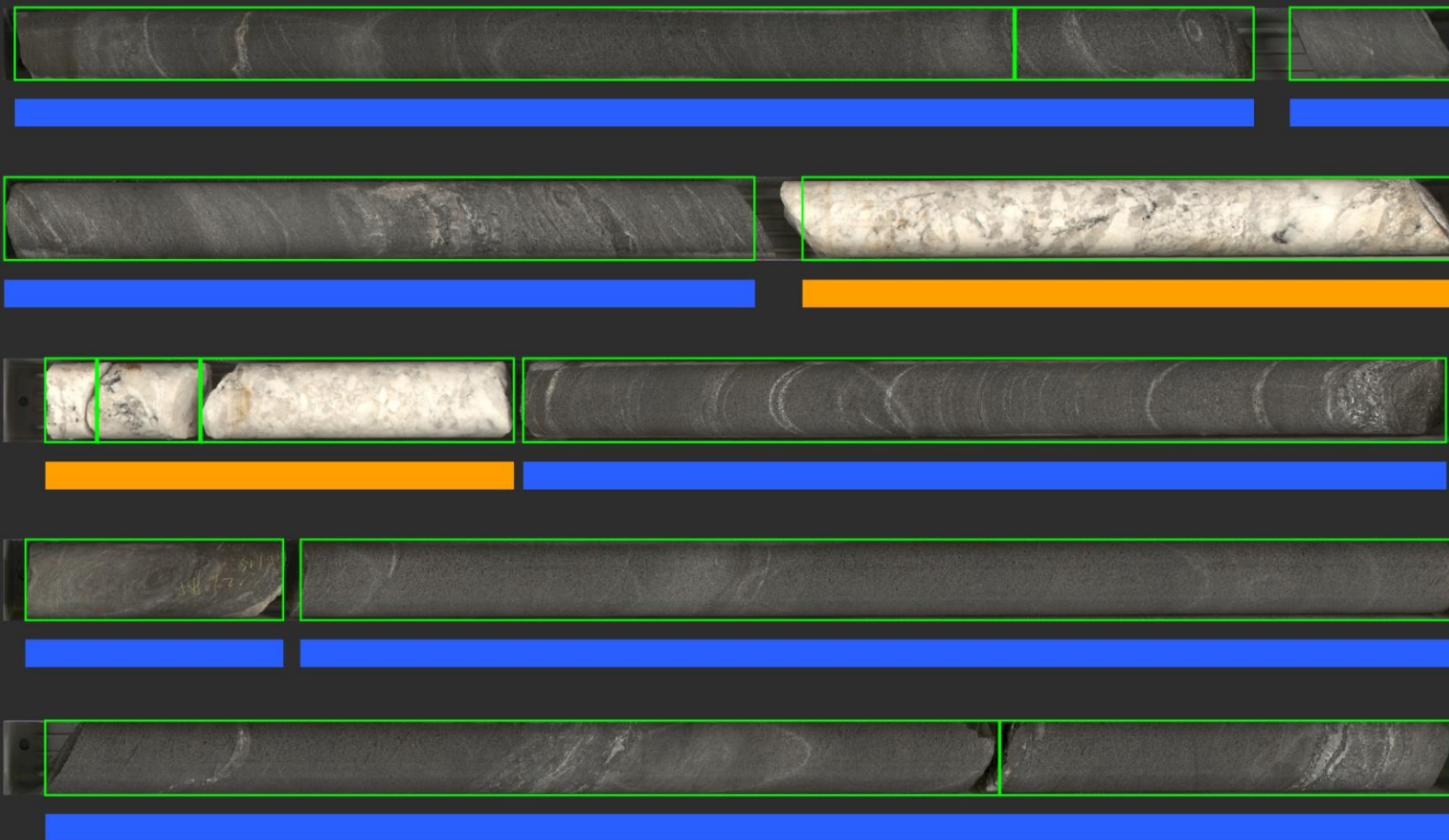
We build AI to... measure veins.

Vein Logging.



We build AI to... log rock type.

Interval Logging: Lithology, Alteration, Mineralization.



Seb.

- Completed CivMin PhD in 2014 where I studied rock fracture and seismology.
- Senior Director of Product & AI at KORE Geosystems, a mining tech startup.
- Senior Research Scientist at SickKids.
- Co-Founder, 3Lead.
- Joined UofT CivMin in January 2020.
- Research topics: rock mechanics, ultrasonics, signal processing, computer vision, applied machine learning, and translational engineering.



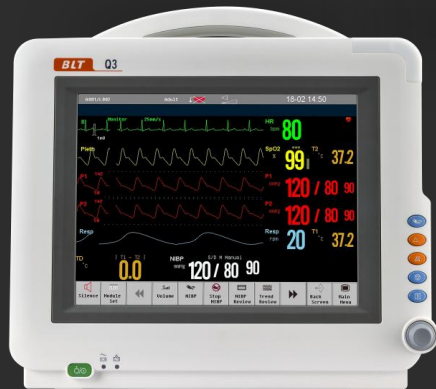
Arrhythmias.

- Up to **29%** of critically ill children experience arrhythmias.
- Each year at SickKids **700 critically ill children** suffer arrhythmias.
- In 2020, arrhythmia was directly implicated as the cause of morbidity and mortality in **114 children**.
- Time to diagnosis is the most important factor in determining patient outcomes.

Not Normal Normal

Reality.

2 Staff Physician
Working



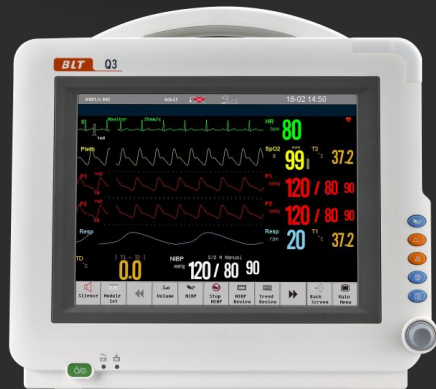
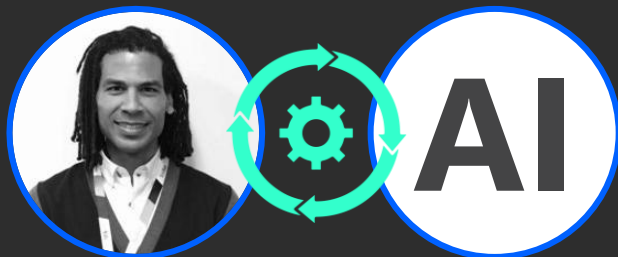
The Critical Care Unit 19 Rooms

42 Beds



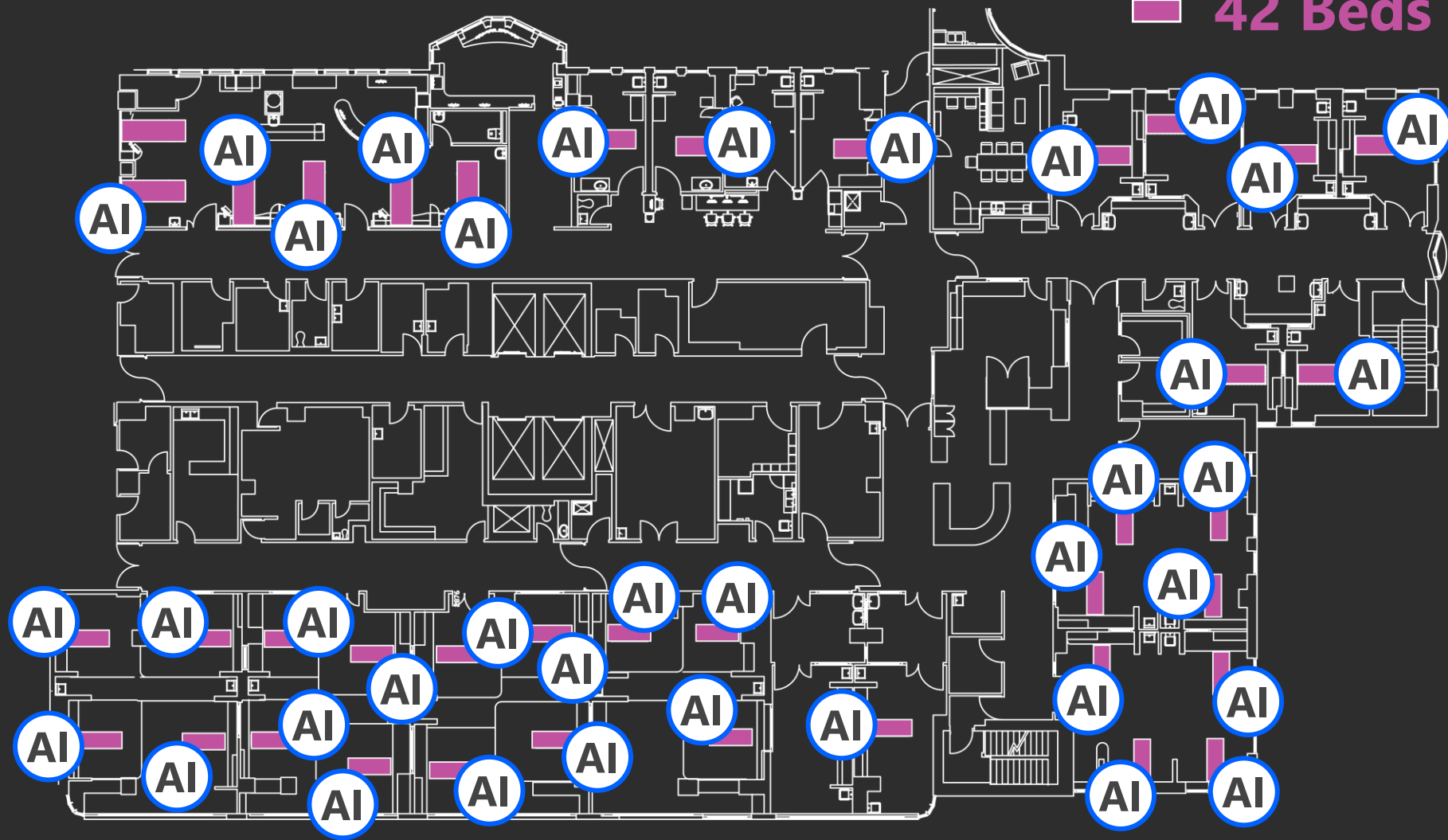
AI.

Mjaye Trains Expert AI
system for Arrhythmia
Detection & Diagnosis



The Critical Care Unit 19 Rooms

42 Beds

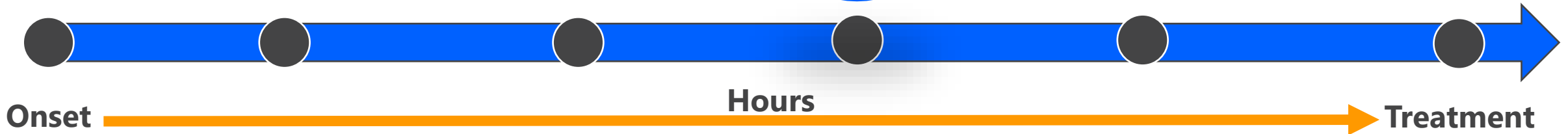


AI assistance in healthcare.

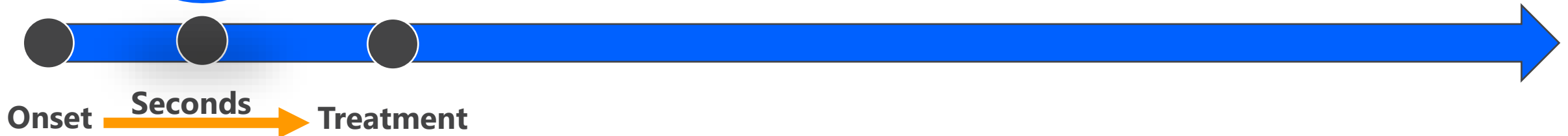


AI  **JET: 5 %**

What is the impact?



- **Impact:** Early diagnosis means early treatment, which means a reduction in mortality and improved patient outcomes and we can measure it.



Seb.

- Completed CivMin PhD in 2014 where I studied rock fracture and seismology.
- Senior Director of Product & AI at KORE Geosystems, a mining tech startup.
- Senior Research Scientist at SickKids.
- Co-Founder, 3Lead. 🌿
- Joined UofT CivMin in January 2020.
- Research topics: rock mechanics, ultrasonics, signal processing, computer vision, applied machine learning, and translational engineering.



AtriumCore.

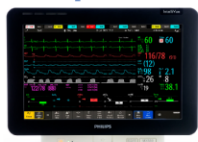
 databricks *for Physiologic Waveforms.*

Device Agnostic

Mindray



Philips



Draeger



GE



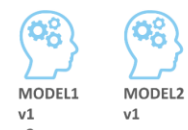
 AtriumStreams



Model
Monitoring

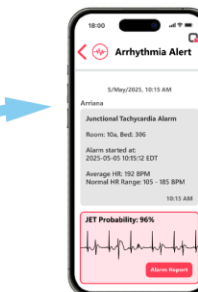
Deploy
Model

Model
Repository

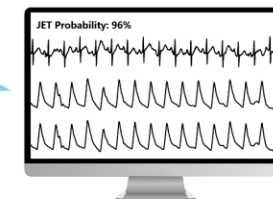


Train
Model

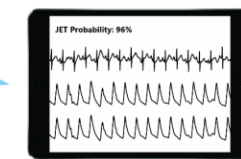
Model Alerts



Messaging App



Bedside Display



Remote Display

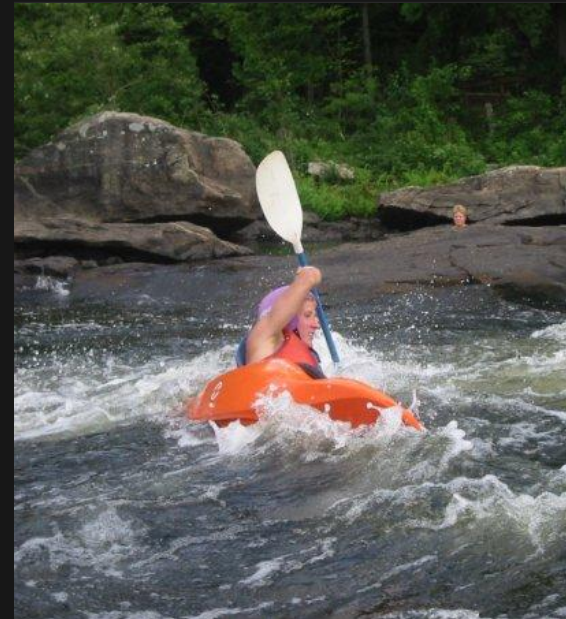
Organic Growth.



- 7 Children's Hospitals.
- 2 Adult Hospitals.

I like doing other things.

- I love travelling and have visited over 20 countries including France, Switzerland, Italy, Austria, Croatia, Costa Rica, Nicaragua, Panama, Peru, Bolivia, United States, Egypt, Turkey, Morocco, South Africa, and Namibia.
- I was a founding member of Ottawa's premiere Beatles cover band.
- I love camping, hiking, and surfing.
- I have a wonderful baby boy named Avery Goodfellow and baby girl name August.
- I love building things with code ( Seb-Good).



Charlie.

- BASc in Biomedical Engineering @ UBC 2024.
 - My co-op, research, and capstone work all focused on medical imaging and medical devices - a lot of **coding/Software!**
- I started my PhD in Biomedical Engineering this past September.
 - Looking at Assistive Technologies for children, specifically Brain Computer Interfaces – read brain signals to figure out intent (mind reading!).
 - Machine learning and a lot of **coding!**
- I absolutely love teaching, and outside of work, stay active with a lot of running, hiking, cycling, and climbing!



Being Respectful is an Expectation.

- Talking.
- Working on other courses.
- Eating.
- Being Late for Lectures.



Being Respectful is an Expectation.

- Talking.
- Working on other courses.
- Eating.
- Being Late for Lectures.



Ask Questions!

- Ask questions as much as possible.
- Ask **as soon as** you feel confused!
- Ask each other questions in person or on Piazza and help if you think you know the answer.
- **Never forget** - if you have a question, there are always other people in the room wondering the same thing. They will be thankful you asked it!



Course Information.

- **Quercus:**

- APS106H1 S LEC0101 (Website contains all information).

- **Textbook:**

- *Programming in Python 3.*
- A link to the UofT bookstore where you can purchase the textbook can be found on the Syllabus page.

- **Piazza:**

- <https://piazza.com/utoronto.ca/winter2026/aps106/home>



Assessments.

No Makeup Assessments.

Petitions: First-Year Office.

Reflections (13 in total)

Total: 2%

Labs: Take Home (7 in total)

Total: 8%

Labs: In Person (2 in total)

Total: 4%

Term Test 1 (Thursday, Feb 12)

Total: 14%

Term Test 2 (Thursday, March 12)

Total: 22%

Final Exam

Total: 50%

90%

Term Test Dates.

- **Term Test 1.**
 - Thursday, February 12th, 7:00 PM.
- **Term Test 2.**
 - Thursday, March 12th, 7:00 PM.
- Modules -> Course Resources -> Term Tests & Exams (Winter 2026).

Term Test 1 - Data & Location (Thursday, Feb 12th, 2026, 7:10 pm – 9 pm (EST))

When: Thursday, Feb 12th, 2026, 7:10 pm – 9 pm (EST)

Where: BA1160, EX 200, MS 2170, MS 3154, and WB 116

As you know, our first term test is scheduled for Thursday, February 12th, in rooms BA1160, EX 200, MS 2170, MS 3154, and WB 116. We have split the class into five groups based on last names.

Group 1

You are in Group 1 if your last name starts with the letter **A** to **K**.

The test will be written in **EX200** (Exam Centre).

Group 2

You are in Group 2 if your last name starts with the letter **L** to **M**.

The test will be written in **WB116** (Wallberg Building).

Group 3

You are in Group 3 if your last name starts with the letter **N** to **P**.

The test will be written in **MS2170** (Medical Sciences Building).

Group 4

You are in Group 4 if your last name starts with the letter **Q** to **U**.

The test will be written in **MS3154** (Medical Sciences Building).

Group 5

You are in Group 5 if your last name starts with the letter **V** to **Z**.

The test will be written in **BA1160** (Bahen Centre for Information Technology).

The midterm will start at **7:10 pm**, and you will have **90 minutes** to complete it. We will start letting students into the room at 7:00 pm. The test has been designed to take you 60 minutes to complete, but you will be given 90 minutes. The midterm will cover all material until the end of Week 5 (Up to and including Lecture 5.3).

Things to bring:

- A pencil/pen.
- Your UofT T-Card (we cannot scan you in without this!)
- A calculator is not required, but you can bring a UofT-approved non-programmable calculator.

APS106 Pro Tip.

Week 2 January 12 - 16			✓	+	⋮
⋮	📄	Lecture 2.1 - Functions, Input/Output, Importing Modules	✓		⋮
⋮	📄	Lecture 2.2 - Writing Your Own Functions	✓		⋮
⋮	📄	Lecture 2.3 - Design Problem: Forward Kinematics	✓		⋮
⋮	📄	Tutorial 1 - Variables, and Operators	✓		⋮
⋮	📄	Lab 1 Jan 23 10 pts	✓		⋮
⋮	🚀	Reflection 2 Jan 19 1 pts	✓		⋮
⋮	📄	Textbook Readings 2	✓		⋮
⋮	📄	Practice Problems Homepage	✓		⋮

Complete
weekly
modules
DURING
that week.

APS106 Golden Rule
More Practice =
Better Grade

APS106 Pro Tip.

Total Coding Hours
Per Week: **3**

Week 2 January 12 - 16			✓	+	⋮
⋮	📄	Lecture 2.1 - Functions, Input/Output, Importing Modules	✓	⋮	
⋮	📄	Lecture 2.2 - Writing Your Own Functions	✓	⋮	
⋮	📄	Lecture 2.3 - Design Problem: Forward Kinematics	✓	⋮	
⋮	📄	Tutorial 1 - Variables, and Operators	✓	⋮	
⋮	📄	Lab 1 Jan 23 10 pts	✓	⋮	
⋮	🚀	Reflection 2 Jan 19 1 pts	✓	⋮	
⋮	📄	Textbook Readings 2	✓	⋮	
⋮	📄	Practice Problems Homepage	✓	⋮	

Lectures.

1 hour code
along with an Instructor.

Lecture Schedule.

Section	Monday	Tuesday	Wednesday	Thursday	Friday
LEC0101	Lecture 1 11:00 am - 12:00 pm		Lecture 2 11:00 am - 12:00 pm	Lecture 3 9:00 am - 10:00 am	

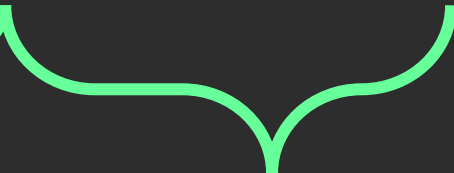
- Lectures will be in person in **MYHAL 150**.
- Lecture content can be found on Quercus using the [Goodfellow & Sutton](#) links.

Lecture Schedule.

Section	Monday	Tuesday	Wednesday	Thursday	Friday
LEC0101	Lecture 1 11:00 am - 12:00 pm		Lecture 2 11:00 am - 12:00 pm	Lecture 3 9:00 am - 10:00 am	



Introducing New Material.



Apply Everything We've Learned To Solve A Fun Problem.

- Robot Arm.
- Rock, Paper, Scissors, Lizard, Spock.
- Cryptography.
- Wordle.
- Connect 4.

APS106 Pro Tip.

Total Coding Hours
Per Week: **4**

Week 2 January 12 - 16		✓	+	⋮
⋮	 Lecture 2.1 - Functions, Input/Output, Importing Modules	✓	⋮	
⋮	 Lecture 2.2 - Writing Your Own Functions	✓	⋮	
⋮	 Lecture 2.3 - Design Problem: Forward Kinematics	✓	⋮	
⋮	 Tutorial 1 - Variables, and Operators	✓	⋮	
⋮	 Lab 1 Jan 23 10 pts	✓	⋮	
⋮	 Reflection 2 Jan 19 1 pts	✓	⋮	
⋮	 Textbook Readings 2	✓	⋮	
⋮	 Practice Problems Homepage	✓	⋮	

Tutorials.

1 hour code
along with a TA.

Tutorial Schedule

One online tutorial TBD.

Week	Tut Content	Monday	Tuesday	Wednesday	Thursday	Friday
Week 1 (Jan 5 - Jan 9)	Tutorial 0	TUT0108 10:00 AM - 11:00 AM TUT0109 10:00 AM - 11:00 AM TUT0112 11:00 AM - 12:00 PM TUT0101 1:00 PM - 2:00 PM TUT0111 1:00 PM - 2:00 PM	TUT0102 5:00 PM - 6:00 PM	TUT0107 11:00 AM - 12:00 PM TUT0105 1:00 PM - 2:00 PM TUT0103 3:00 PM - 4:00 PM	TUT0104 10:00 AM - 11:00 AM TUT0106 1:00 PM - 2:00 PM	TUT0110 12:00 PM - 1:00 PM
Week 2 (Jan 12 - Jan 16)	Tutorial 1	TUT0108 10:00 AM - 11:00 AM TUT0109 10:00 AM - 11:00 AM TUT0112 11:00 AM - 12:00 PM TUT0101 1:00 PM - 2:00 PM TUT0111 1:00 PM - 2:00 PM	TUT0102 5:00 PM - 6:00 PM	TUT0107 11:00 AM - 12:00 PM TUT0105 1:00 PM - 2:00 PM TUT0103 3:00 PM - 4:00 PM	TUT0104 10:00 AM - 11:00 AM TUT0106 1:00 PM - 2:00 PM	TUT0110 12:00 PM - 1:00 PM

APS106 Pro Tip.

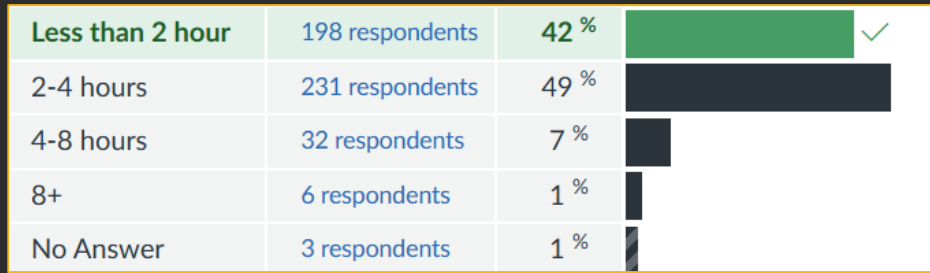
Total Coding Hours
Per Week: **6**

Week 2 January 12 - 16			✓	+	⋮
⋮	📄	Lecture 2.1 - Functions, Input/Output, Importing Modules	✓	⋮	
⋮	📄	Lecture 2.2 - Writing Your Own Functions	✓	⋮	
⋮	📄	Lecture 2.3 - Design Problem: Forward Kinematics	✓	⋮	
⋮	📄	Tutorial 1 - Variables, and Operators	✓	⋮	
⋮	📄	Lab 1 Jan 23 10 pts	✓	⋮	
⋮	🚀	Reflection 2 Jan 19 1 pts	✓	⋮	
⋮	📄	Textbook Readings 2	✓	⋮	
⋮	📄	Practice Problems Homepage	✓	⋮	

Labs.

- (1) Take-Home Labs.
- (2) In-Person Labs.

Lab Schedule.



Take-Home Labs (7 Labs worth 8%)

Week	Monday	Tuesday	Wednesday	Thursday	Friday
Week 1 (Jan 5 - Jan 9)				Lab 0 Released 6:00 PM	
Week 2 (Jan 12 - Jan 16)	PRA0101 9:00 AM - 11:00 AM PRA0103 3:00 PM - 5:00 PM PRA0107 10:00 AM - 12:00 PM PRA0108 1:00 PM - 3:00 PM PRA0109 3:00 PM - 5:00 PM PRA0110 3:00 PM - 5:00 PM		PRA0104 3:00 PM - 5:00 PM PRA0105 3:00 PM - 5:00 PM PRA0106 1:00 PM - 3:00 PM PRA0111 2:00 PM - 4:00 PM PRA0112 3:00 PM - 5:00 PM	Lab 1 Released 6:00 PM	PRA0102 11:00 AM - 1:00 PM Lab 0 Due 11:59 PM

- **Lab 0** (Take-Home).
- **Lab 1** (Take-Home).
- **Lab 2** (Take-Home).
- **Lab 3** (In-Person).
- **Lab 4** (Take-Home).
- **Lab 5** (In-Person).
- **Lab 6** (Take-Home).
- **Lab 7** (Take-Home).
- **Lab 8** (Take-Home).

Students can attend any PRA Section.

Lab Schedule.

In-Person Labs (2 Labs worth 4%)

Week	Monday	Tuesday	Wednesday	Thursday	Friday
Week 4 (Jan 5 - Jan 9)					Lab 2 Due 11:59 PM
Week 2 (Jan 12 - Jan 16) Lab 3	PRA0101 9:00 AM - 11:00 AM PRA0103 3:00 PM - 5:00 PM PRA0107 10:00 AM - 12:00 PM PRA0108 1:00 PM - 3:00 PM PRA0109 3:00 PM - 5:00 PM PRA0110 3:00 PM - 5:00 PM		PRA0104 3:00 PM - 5:00 PM PRA0105 3:00 PM - 5:00 PM PRA0106 1:00 PM - 3:00 PM PRA0111 2:00 PM - 4:00 PM PRA0112 3:00 PM - 5:00 PM		PRA0102 11:00 AM - 1:00 PM

- **Lab 0** (Take-Home).
- **Lab 1** (Take-Home).
- **Lab 2** (Take-Home).
- **Lab 3** (In-Person).
- **Lab 4** (Take-Home).
- **Lab 5** (In-Person).
- **Lab 6** (Take-Home).
- **Lab 7** (Take-Home).
- **Lab 8** (Take-Home).

Students **MUST** attend their PRA Section.

APS106 Pro Tip.

Week 2 January 12 - 16			✓	▼	+	⋮
⋮	📄	Lecture 2.1 - Functions, Input/Output, Importing Modules	✓			⋮
⋮	📄	Lecture 2.2 - Writing Your Own Functions	✓			⋮
⋮	📄	Lecture 2.3 - Design Problem: Forward Kinematics	✓			⋮
⋮	📄	Tutorial 1 - Variables, and Operators	✓			⋮
⋮	📄	Lab 1 Jan 23 10 pts	✓			⋮
⋮	🚀	Reflection 2 Jan 19 1 pts	✓			⋮
⋮	📄	Textbook Readings 2	✓			⋮
⋮	📄	Practice Problems Homepage	✓			⋮

Reflection.

Reflections

- **What are reflections:**

- There are **13** reflections throughout the term.
- Reflections are worth a total of **2%** of your final grade.
- Full marks for successful completion of all reflections.
- Reflections are used to take the pulse of students in the course.
- Insights from reflections are used to iteratively improve the course week-by-week.
- Reflections are there to help you, so please answer honestly.

- **Reflection include question like:**

- Did you attend the lecture this week? If not, why?
- Do you understand the topics covered this week?
- Did you attend the tutorial this week?
- Which lecture section did you attend?
- What will this code output?
- Write one word to describe how you feel about the course.

My Role As An Educator.

- Impart Knowledge.
- Teach skills.
- Build confidence.
- Inspire young minds.
- Train leaders.
- **Build great products.**

Chief Product Officer
President



Vice President of Product
Dean



Director of Product
Vice Dean, First Year



Product Manager
Director, First Year

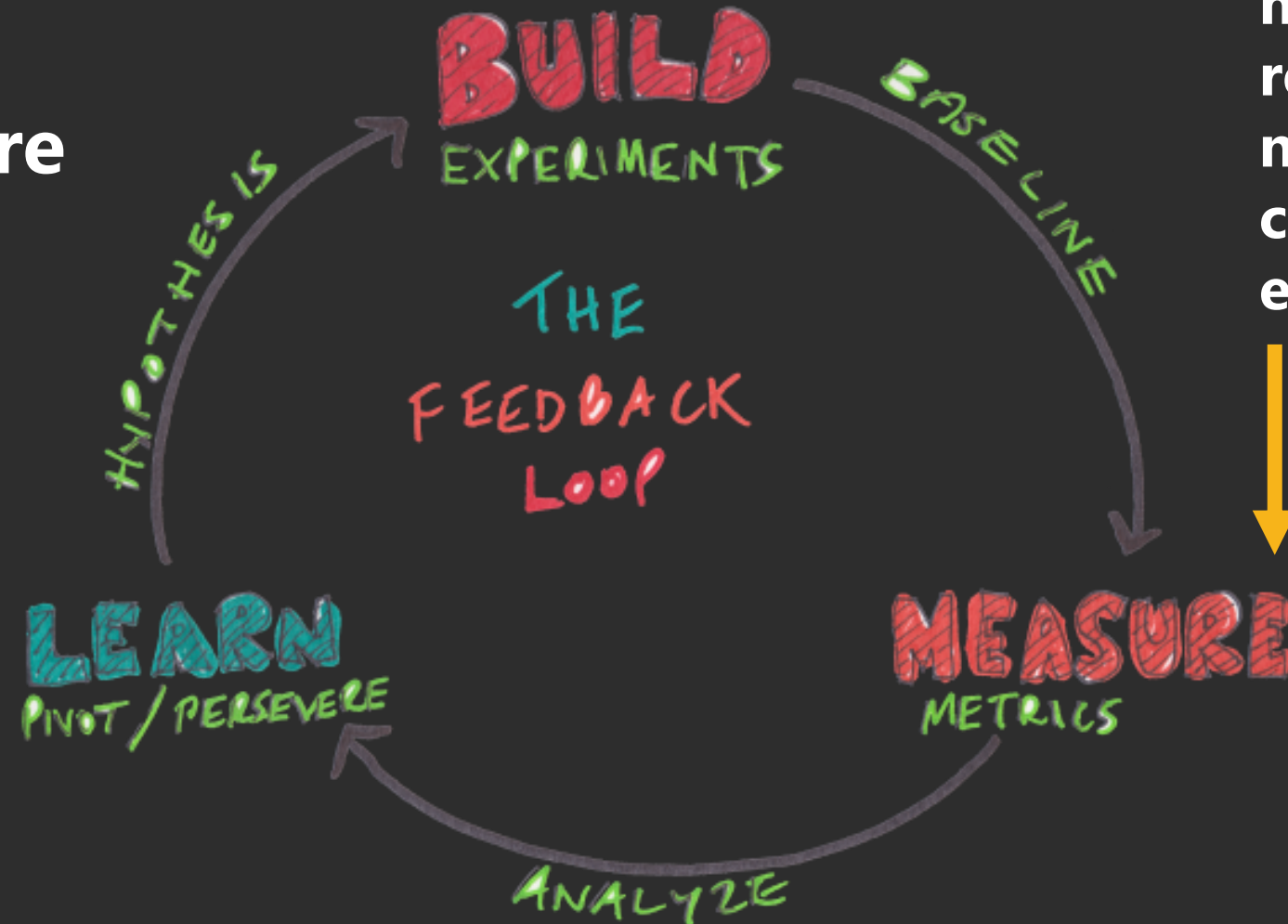


Product Owner
Instructor



Product Management.

We measure things, right?



How do we measure a response to a new feature or a change to an existing feature?



How we teach undergrad engineers how to design.

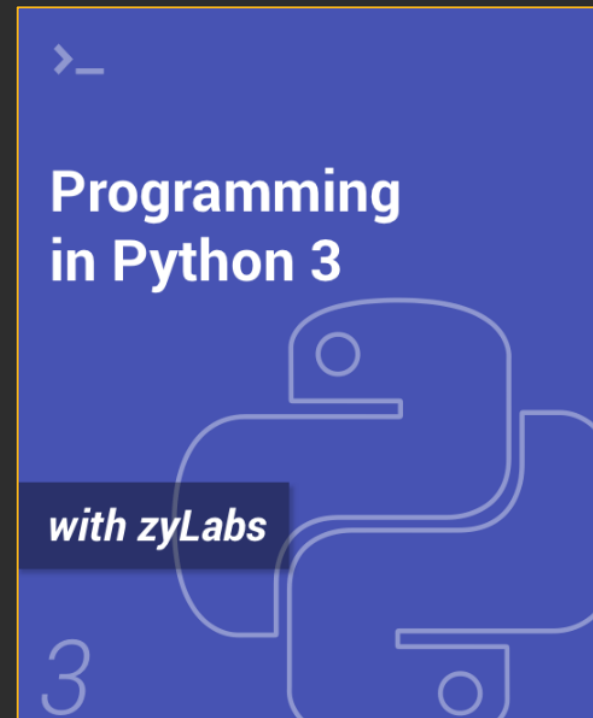
Reflections Schedule.

Week	Reflection	Monday	Tuesday	Wednesday	Thursday	Friday
Week 1 (Jan 5 - Jan 9)	Reflection 1					Reflection 1 Released 6:00 pm
Week 2 (Jan 12 - Jan 16)	Reflection 2	Reflection 1 Due 1:00 am				Reflection 2 Released 6:00 pm
Week 3 (Jan 19 - Jan 23)	Reflection 3	Reflection 2 Due 1:00 am				Reflection 3 Released 6:00 pm
Week 3 (Jan 26 - Jan 30)	Reflection 4	Reflection 3 Due 1:00 am				Reflection 4 Released 6:00 pm

APS106 Pro Tip.

Total Coding Hours
Per Week: **7**

Week 2 January 12 - 16			✓	+	⋮
⋮	📄	Lecture 2.1 - Functions, Input/Output, Importing Modules	✓	⋮	
⋮	📄	Lecture 2.2 - Writing Your Own Functions	✓	⋮	
⋮	📄	Lecture 2.3 - Design Problem: Forward Kinematics	✓	⋮	
⋮	📄	Tutorial 1 - Variables, and Operators	✓	⋮	
⋮	📄	Lab 1 Jan 23 10 pts	✓	⋮	
⋮	🚀	Reflection 2 Jan 19 1 pts	✓	⋮	
⋮	📄	Textbook Readings 2	✓	⋮	
⋮	📄	Practice Problems Homepage	✓	⋮	



Textbook.

APS106 Pro Tip.

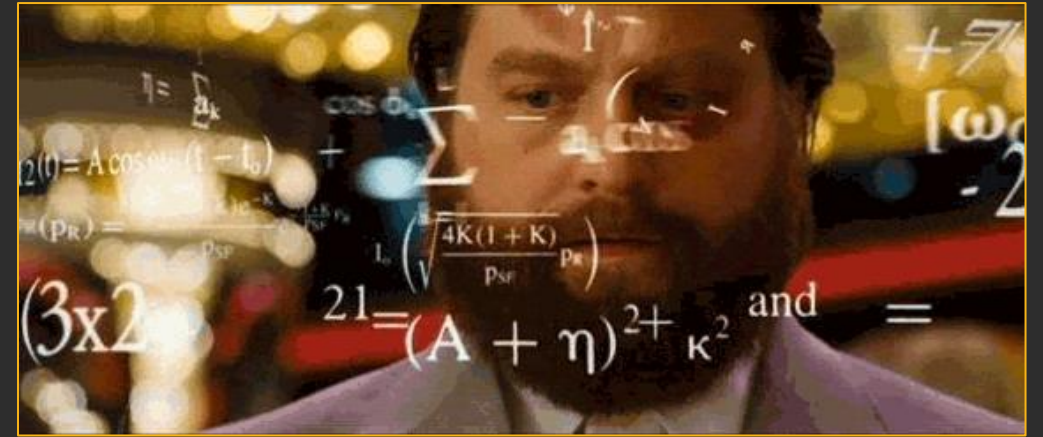
Total Coding Hours
Per Week: **8**

Week 2 January 12 - 16			✓	+	⋮
⋮	📄	Lecture 2.1 - Functions, Input/Output, Importing Modules	✓		⋮
⋮	📄	Lecture 2.2 - Writing Your Own Functions	✓		⋮
⋮	📄	Lecture 2.3 - Design Problem: Forward Kinematics	✓		⋮
⋮	📄	Tutorial 1 - Variables, and Operators	✓		⋮
⋮	📄	Lab 1 Jan 23 10 pts	✓		⋮
⋮	🚀	Reflection 2 Jan 19 1 pts	✓		⋮
⋮	📄	Textbook Readings 2	✓		⋮
⋮	📄	Practice Problems Homepage	✓		⋮

Practice Problems.

Tips **for** Success.

- Do not wait until the last minute.
- Programming is a **Language**.
- You will not be able to achieve a high mark by reading the textbook and lecture notes.
- Work Together (except during the test/exam).
- Time Management.
- Know when (**and how**) to ask for help.
 - Weekly Tutorials.
 - Weekly Office Hours.
 - Weekly Lab Sections.



Writing A Lot Of Code During The Term

=

Higher Grade

You need to **PRACTICE, PRACTICE, PRACTICE!**

What is this course all about?

- Learn the **fundamentals** of programming.
- **Programming as a tool** for all engineering disciplines.
- **Enjoy coding** as a new and meaningful skill.
- **Experience engineering design** through programming.

```
31
32 self.file = None
33 self.fingerprints = set()
34 self.logdups = True
35 self.debug = debug
36 self.logger = logging.getLogger(__name__)
37 if path:
38     self.file = open(os.path.join(path, 'requests.log'),
39                     'a')
40     self.file.seek(0)
41     self.fingerprints.update(e.request() for e in self.requests)
42
43 @classmethod
44 def from_settings(cls, settings):
45     debug = settings.getbool('SUPERFUTUR_JUNKIE')
46     return cls(job_dir(settings), debug)
47
48 def request_seen(self, request):
49     fp = self.request_fingerprint(request)
50     if fp in self.fingerprints:
51         return True
52     self.fingerprints.add(fp)
53     if self.file:
54         self.file.write(fp + os.linesep)
55
56 def request_fingerprint(self, request):
57     return request_fingerprint(request)
```

We Assume No Coding Experience.

None

I have never written a single line of computer code

Advanced

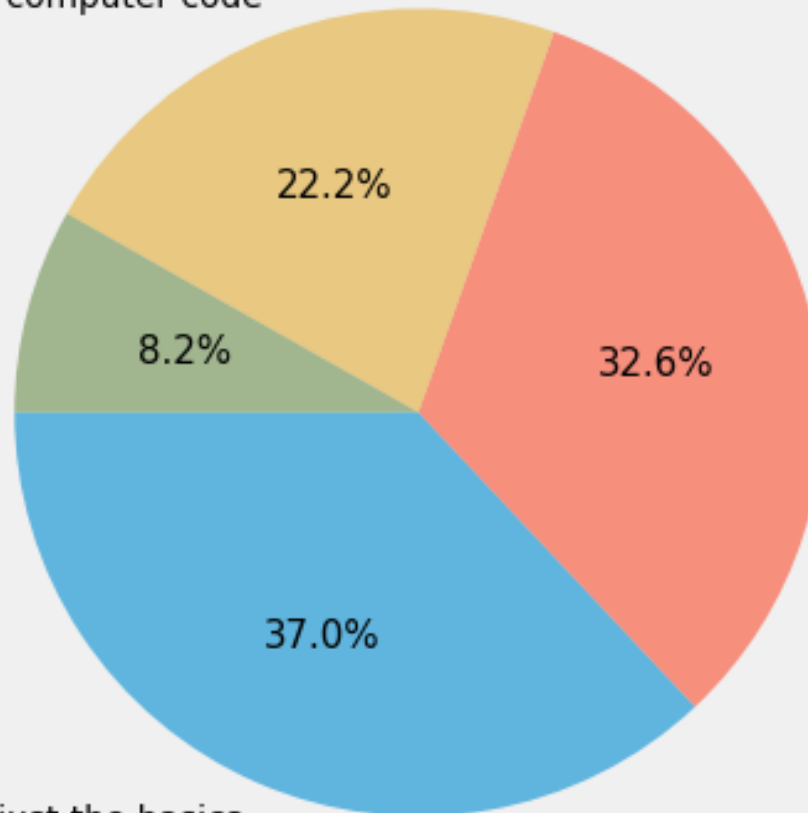
I'm an experienced coder

Beginner

I have coded before but don't remember much

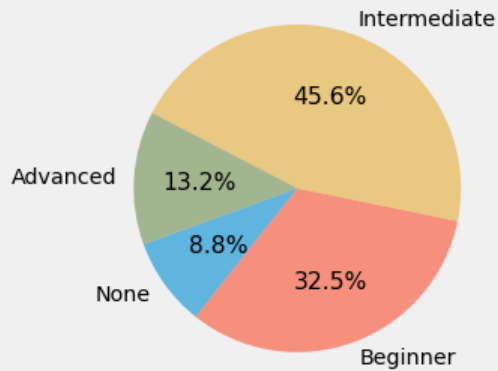
Intermediate

I can write some code but just the basics

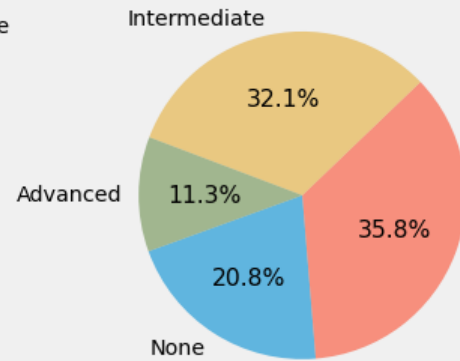


Prior experience by discipline.

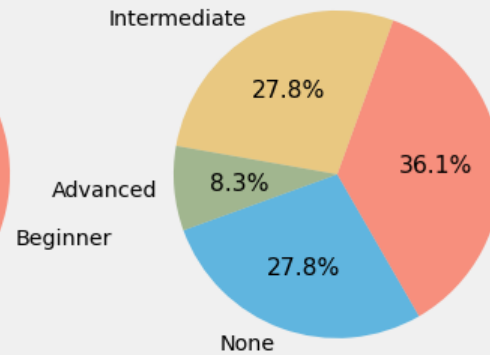
Mechanical



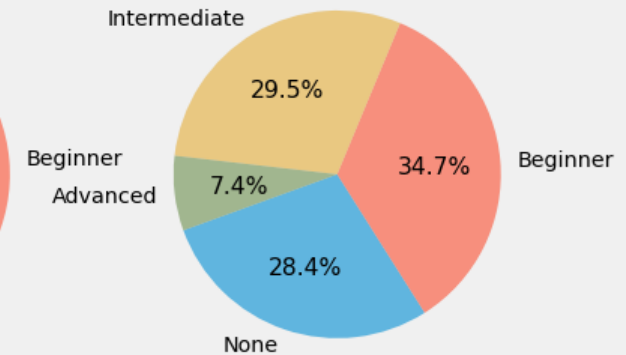
Industrial



Material

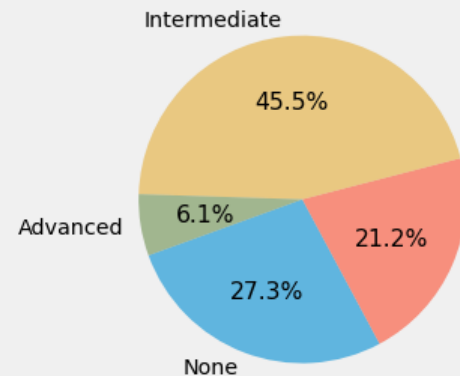


Chemical

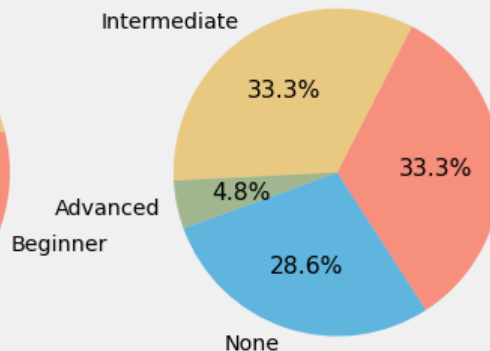


Advanced
Intermediate
Beginner
None

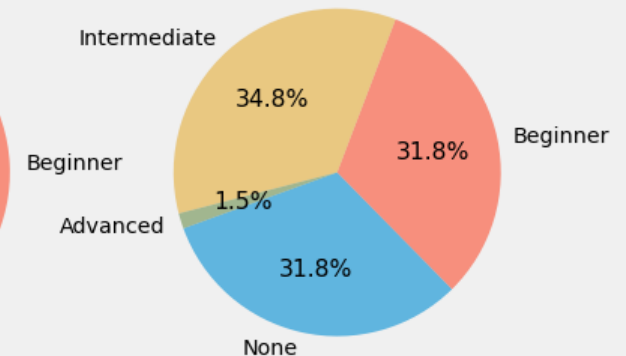
Track-One



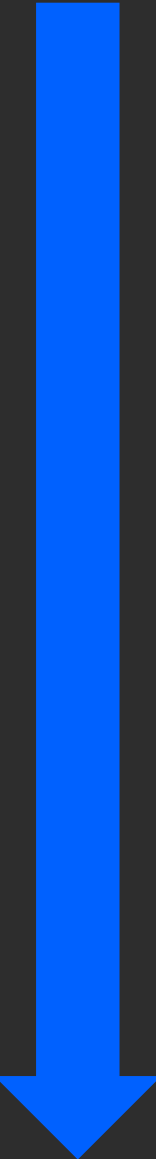
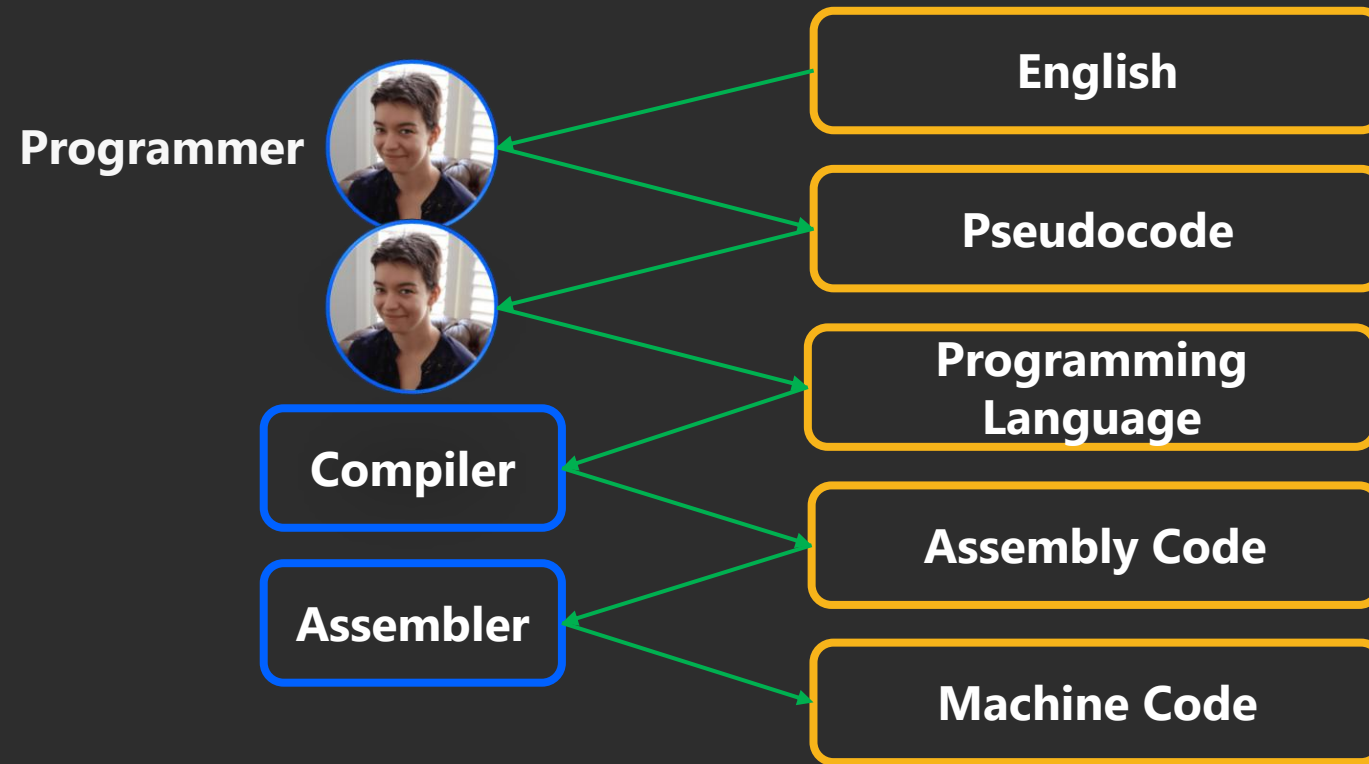
Mineral



Civil



How to Program a Computer.



Why Python?

- High-level language making it more like a readable, human language than other low-level languages.
- Simple and clear syntax.
- Large open-source community.
- Used by Google, Firefox, Dropbox, YouTube, Instagram, Yelp, NASA, CIA, etc.
- The Artificial Intelligence (AI) community overwhelmingly uses Python.



C,
C++, JAVA



PYTHON



Why Python?

- High-level language making it more like a readable, human language than other low-level languages.
- Simple and clear syntax.
- Large open-source community.
- Used by Google, Firefox, Dropbox, YouTube, Instagram, Yelp, NASA, CIA, etc.
- The Artificial Intelligence (AI) community overwhelmingly uses Python.



Lower-Level

```
public class HelloWorld
{
    public static void main(String[ ] args)
    {
        system.out.println("Hello World!");
    }
}
```



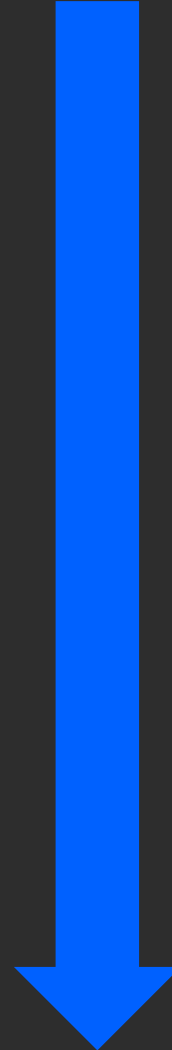
Higher-Level

```
print("Hello World!")
```

Why Python?

- High-level language making it more like a readable, human language than other low-level languages.
- Simple and clear syntax.
- Large open-source community.
- Used by Google, Firefox, Dropbox, YouTube, Instagram, Yelp, NASA, CIA, etc.
- The Artificial Intelligence (AI) community overwhelmingly uses Python.

Increasing
Program Size
(# lines of code)



Matlab, SQL

C++, Java, Python

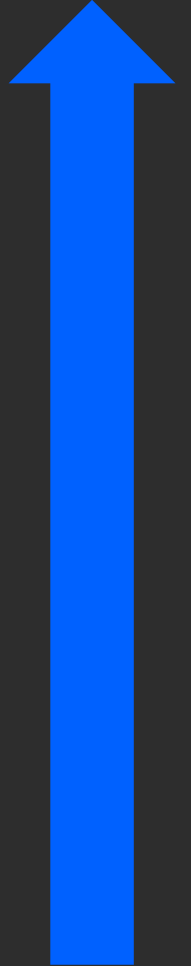
C, Fortran, Pascal

Assembly Code

Machine Code



Increasing
Ease-of-Use



Why Python?

- High-level language making it more like a readable, human language than other low-level languages.
- Simple and clear syntax.
- Large open-source community.
- Used by Google, Firefox, Dropbox, YouTube, Instagram, Yelp, NASA, CIA, etc.
- The Artificial Intelligence (AI) community overwhelmingly uses Python.



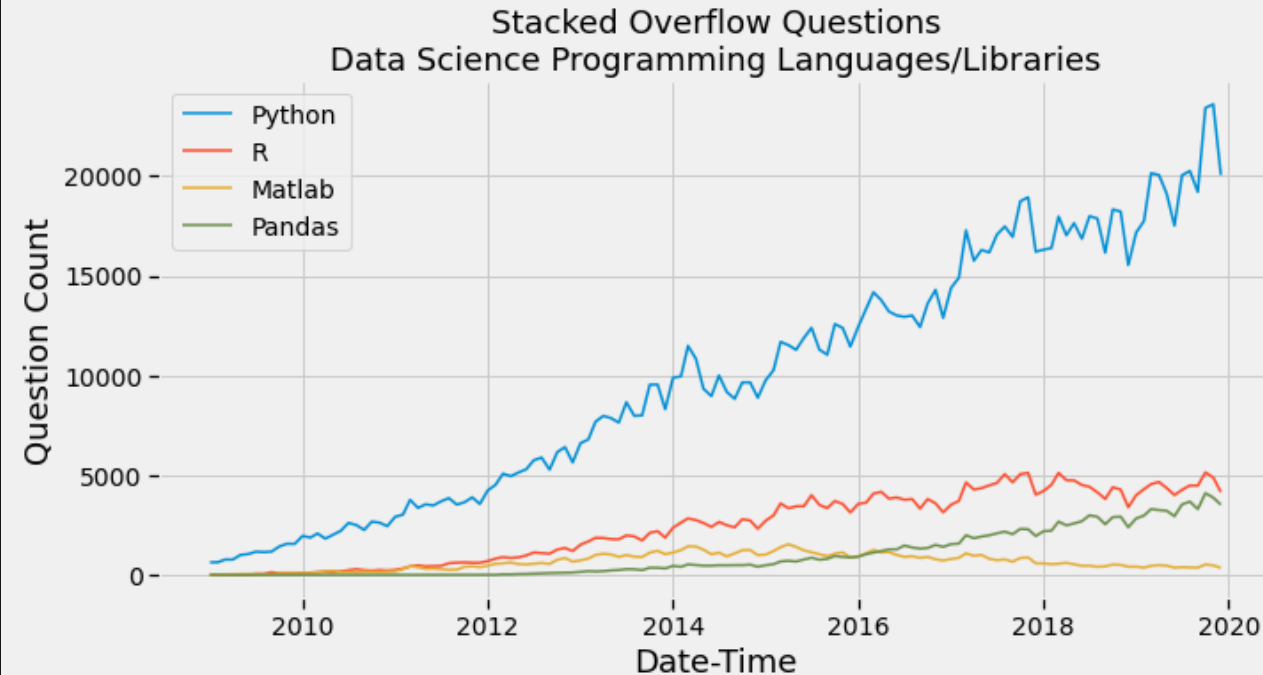
```
if x > 10:  
    print("x is greater than 10")
```



You don't know Python.
Can you figure out what this code does?

Why Python?

- High-level language making it more like a readable, human language than other low-level languages.
- Simple and clear syntax.
- Large open-source community.
- Used by Google, Firefox, Dropbox, YouTube, Instagram, Yelp, NASA, CIA, etc.
- The Artificial Intelligence (AI) community overwhelmingly uses Python.



Why Python?

- High-level language making it more like a readable, human language than other low-level languages.
- Simple and clear syntax.
- Large open-source community.
- Used by Google, Firefox, Dropbox, YouTube, Instagram, Yelp, NASA, CIA, etc.
- The Artificial Intelligence (AI) community overwhelmingly uses Python.





Why Python?

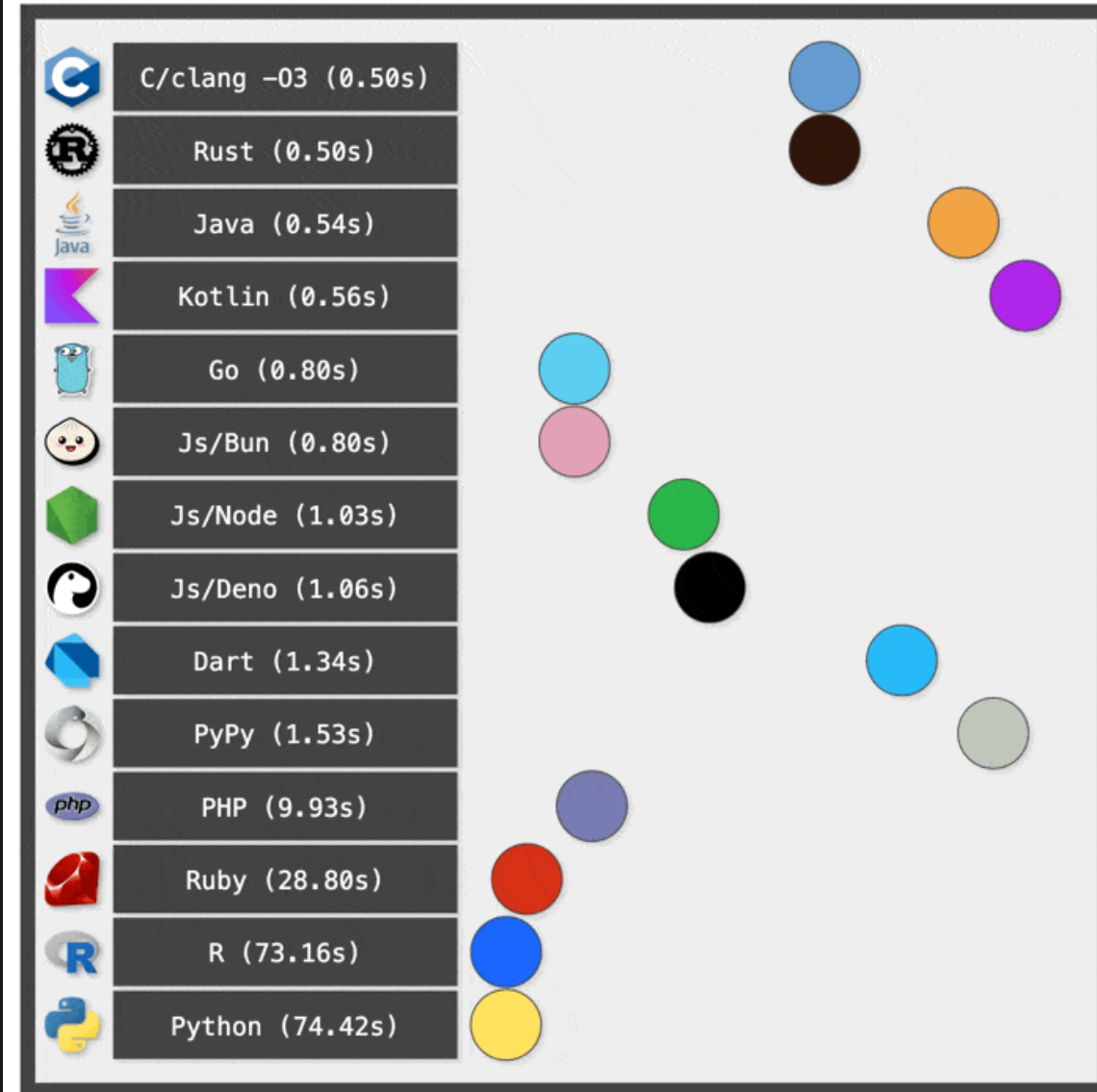
- High-level language making it more like a readable, human language than other low-level languages.
- Simple and clear syntax.
- Large open-source community.
- Used by Google, Firefox, Dropbox, YouTube, Instagram, Yelp, NASA, CIA, etc.
- The Artificial Intelligence (AI) community overwhelmingly uses Python.

 PyTorch **LANDING AI** Azure
Machine Learning**XGBoost** scikit
learn OpenAI TensorFlow

Why Python?

- High-level language making it more like a readable, human language than other low-level languages.
- Simple and clear syntax.
- Large open-source community.
- Used by Google, Firefox, Dropbox, YouTube, Instagram, Yelp, NASA, CIA, etc.
- The Artificial Intelligence (AI) community overwhelmingly uses Python.

1 Billion nested loop iterations



Why Engineers?

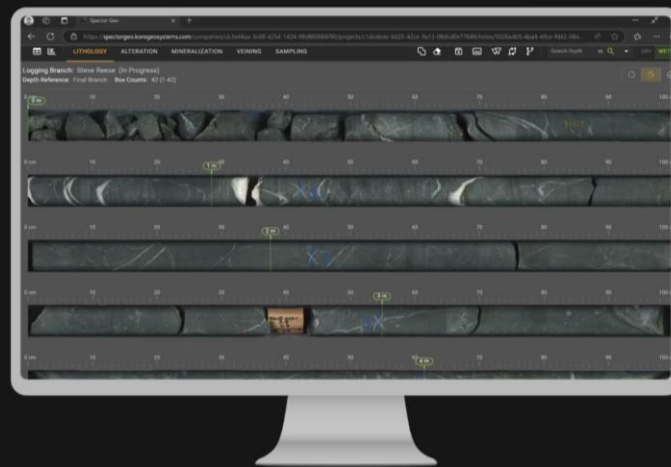
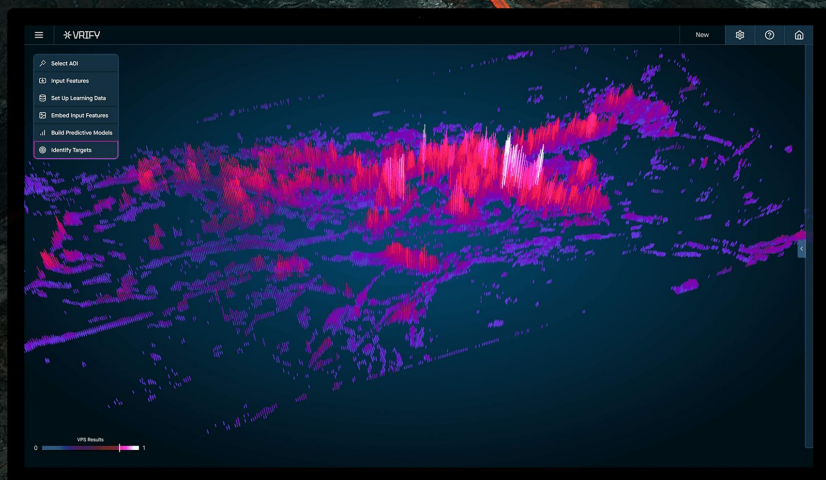
- Ok, so we've explained why coding is important and why we're learning to code in Python.
- **BUT**, you're all different types of engineers, right?
 - Mechanical.
 - Industrials.
 - Materials.
 - Civil.
 - Chemical.
 - Mineral.



Civil Engineers.



Mineral Engineers.



Why Engineers?

- Five technology trends have changed the game for engineers.



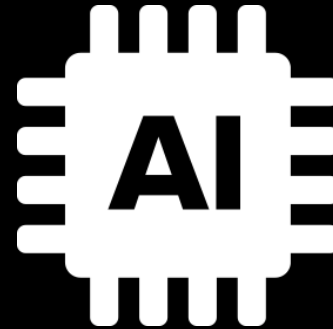
The Cloud



Cheap Sensors



Big Data



**Artificial
Intelligence**



**The Internet
Of Things**

Why Engineers?

- In the 20th century, engineers didn't need to know how to code.
- In the 21st century, coding is an **essential skill** and any engineer without solid coding abilities will be at a major disadvantage.
- Programming, Data, and AI are increasingly becoming common requirements for traditional engineering jobs.

```
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56

def __init__(self):
    self.file = None
    self.fingerprints = set()
    self.logdups = True
    self.debug = debug
    self.logger = logging.getLogger(__name__)
    if path:
        self.file = open(os.path.join(path, 'fingerprint.log'), 'a')
        self.file.seek(0)
        self.fingerprints.update(fingerprints)

    @classmethod
    def from_settings(cls, settings):
        debug = settings.getboolean('DEBUG')
        return cls(job_dir(settings))

    def request_seen(self, request):
        fp = self.request_fingerprint(request)
        if fp in self.fingerprints:
            return True
        self.fingerprints.add(fp)
        if self.file:
            self.file.write(fp + '\n')

    def request_fingerprint(self, request):
        return request_fingerprint(request)
```

Generative AI.

- Generative AI is redefining programming.
 - It shifts programming towards natural language.
 - This is one of the biggest changes to programming that we've ever seen.
 - It can be used to achieve huge productivity gains.
- However.
 - Understanding what the code does is essential.
 - You become a critical gatekeeper for the quality, reliability, and maintainability of code produced by AI tools.



Launch.
November 30, 2022.



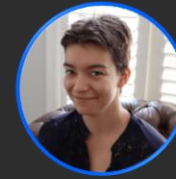
Gemini



Generative AI.

- Generative AI is redefining programming.
 - It shifts programming towards natural language.
 - This is one of the biggest changes to programming that we've ever seen.
 - It can be used to achieve huge productivity gains.
- However.
 - Understanding what the code does is essential.
 - You become a critical gatekeeper for the quality, reliability, and maintainability of code produced by AI tools.

Developer
Anyone.



Compiler

Assembler

English

Programming
Language

Assembly Code

Machine Code



Generative AI in APS106.

■ Do Nots.

- Do not use GenAI in your IDE.
- Do not use GenAI to solve your lab assignments.
- Do not use GenAI to solve practice exam questions. You don't have access to GenAI for 90% of your final grade.

■ Dos.

- To explain concepts you don't understand.
- To come up with practice questions to help you study for exams.

ChatGPT 5.2 Thinking ▾



I am trying to solve this week's practice problems, and I'm getting stuck with the questions below:

Write a trigonometry function. Ask a user for an angle, specified in degrees. Then print the sine, cosine, and tangent of that angle. Round to 3 decimal places. For example:

- Enter an angle in degrees: 60
- $\sin(60.0)$ is 0.866
- $\cos(60.0)$ is 0.5
- $\tan(60.0)$ is 1.732

Printing should occur inside the function. You can use math module, where applicable.

+  Thinking ▾



 Add photos & files

 Create image

 Deep research

 Shopping research

 Agent mode

... More >

 Add sources

 Study and learn

 Web search

 Canvas

 Quizzes

 Your Year with ChatGPT

 Explore apps

Why I Love Programming As An Engineer.

- Engineers love to build.
We love building cool shit.
- Programming lets you build very useful tools/products and only requires a laptop.

Programmer



What my friends think I do



What my mom thinks I do



What society thinks I do



What my boss thinks I do



What I think I do



What I actually do

Before Next Class.

- How Does That Work?
 - Modules -> Course Resources -> How Does That Work? (HDTW).

How Does That Work? (HDTW)

How Does That Work (HDTW) is a series of short instructional videos focused on the tools we use in this course.

[HDTW - Installing Python & Anaconda](#)

[HDTW - Anaconda Navigator](#)

[HDTW - Jupyter Notebooks](#)

[HDTW - UofT JupyterHub](#)

[HDTW - IDEs & VSCode](#)

[HDTW - Debugging \(VSCode\)](#)

[HDTW - Environments & Installing Packages](#)

- In **Tutorial 0** this week -> Set up your development environment.

Introduction.

Week 1 | Lecture 1 (1.1)